5. Copy constructors are needed when you, the programmer, must control precisely how a copy of an object is made. This is important only when the default bitwise copy creates undesired side effects.

REVIEW SKILLS CHECK: Chapter 6

1.
```
class myclass
{
    int x, y;
public:
    myclass(int i, int j) { x=i; y=j; }
    myclass() { x=0; y=0; }
};
```

2.
```
class myclass
{
    int x, y;
public:
    myclass(int i=0, int j=0) { x=i; y=j; }
};
```

3. Once default arguments have begun, a non-defaulting parameter cannot occur.

4. A function cannot be overloaded when the only difference is that one version takes a value parameter and the other takes a reference parameter. (The compiler cannot tell them apart.)

5. It is appropriate to use default arguments when there are one or more values that will occur frequently. Is is inappropriate when there is no value or values that have a greater likelihood of occurring.

6. No, because there is no way to initialize a dynamic array. This class has only one constructor, and it requires initializers.

7. A copy constructor is a special constructor that is called when one object initializes another. This circumstance can occur in any of the following three ways: When one object is explicitly used to initialize another, when an object is passed to a function, or when a temporary object is created as a function return value.

6.2 EXERCISES

1.
```
// Overload the * and / relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
```

```
    coord operator*(coord ob2);
    coord operator/(coord ob2);
};

// Overload * relative to coord class.
coord coord::operator*(coord ob2)
{
    coord temp;

    temp.x = x * ob2.x;
    temp.y = y * ob2.y;

    return temp;
}

// Overload / relative to coord class.
coord coord::operator/(coord ob2)
{
    coord temp;

    temp.x = x / ob2.x;
    temp.y = y / ob2.y;

    return temp;
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3;
    int x, y;

    o3 = o1 * o2;
    o3.get_xy(x, y);
    cout << "(o1*o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1 / o2;
    o3.get_xy(x, y);
    cout << "(o1/o2) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

2. The overloading of the % operator is inappropriate because its operation is unrelated to the traditional use.

6.3 EXERCISE

1.
```
// Overload the < and > relative to coord class.
#include <iostream>
using namespace std;
```

```
class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    int operator<(coord ob2);
    int operator>(coord ob2);
};

// Overload < relative to coord class.
int coord::operator<(coord ob2)
{
    return x<ob2.x && y<ob2.y;
}

// Overload > relative to coord class.
int coord::operator>(coord ob2)
{
    return x>ob2.x && y>ob2.y;
}

int main()
{
    coord o1(10, 10), o2(5, 3);
    int x, y;

    if(o1>o2)
        cout << "o1 > o2\n";
    else
        cout << "o1 <= o2\n";

    if(o1<o2)
        cout << "o1 < o2\n";
    else
        cout << "o1 >= o2\n";

    return 0;
}
```

6.4 EXERCISES

1.
```
// Overload the -- relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
```

```
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator--(); // prefix
    coord operator--(int notused); // postfix
};

// Overload prefix -- for coord class.
coord coord::operator--()
{
    x--;
    y--;
    return *this;
}

// Overload postfix -- for coord class.
coord coord::operator--(int notused)
{
    x--;
    y--;
    return *this;
}

int main()
{
    coord o1(10, 10);
    int x, y;

    o1--; // decrement an object
    o1.get_xy(x, y);
    cout << "(o1--) X: " << x << ", Y: " << y << "\n";

    --o1; // decrement an object
    o1.get_xy(x, y);
    cout << "(--o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

2.
```
// Overload the + relative to coord class.
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator+(coord ob2); // binary plus
```

```
    coord operator+(); // unary plus
};

// Overload + relative to coord class.
coord coord::operator+(coord ob2)
{
    coord temp;

    temp.x = x + ob2.x;
    temp.y = y + ob2.y;

    return temp;
}

// Overload unary + for coord class.
coord coord::operator+()
{
    if(x<0)
        x = -x;
    if(y<0)
        y = -y;

    return *this;
}

int main()
{
    coord o1(10, 10), o2(-2, -2);
    int x, y;

    o1 = o1 + o2; // addition
    o1.get_xy(x, y);
    cout << "(o1+o2) X: " << x << ", Y: " << y << "\n";

    o2 = +o2; // absolute value
    o2.get_xy(x, y);
    cout << "(+o2) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

6.5 EXERCISES

1. ```
   // Overload the - and / relative to coord class.
   #include <iostream>
   using namespace std;

   class coord
   {
       int x, y; // coordinate values
   public:
   ```
   ```

```
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    friend coord operator-(coord ob1, coord ob2);
    friend coord operator/(coord ob1, coord ob2);
};

// Overload - relative to coord class using friend.
coord operator-(coord ob1, coord ob2)
{
    coord temp;

    temp.x = ob1.x - ob2.x;
    temp.y = ob1.y - ob2.y;

    return temp;
}

// Overload / relative to coord class using friend.
coord operator/(coord ob1, coord ob2)
{
    coord temp;

    temp.x = ob1.x / ob2.x;
    temp.y = ob1.y / ob2.y;

    return temp;
}

int main()
{
    coord o1(10, 10), o2(5, 3), o3;
    int x, y;

    o3 = o1 - o2;
    o3.get_xy(x, y);
    cout << "(o1-o2) X: " << x << ", Y: " << y << "\n";

    o3 = o1 / o2;
    o3.get_xy(x, y);
    cout << "(o1/o2) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

2.
```
// Overload the * for ob*int and int*ob.
#include <iostream>
using namespace std;

class coord
{
```

```
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    friend coord operator*(coord ob1, int i);
    friend coord operator*(int i, coord ob2);
};

// Overload * one way.
coord operator*(coord ob1, int i)
{
    coord temp;

    temp.x = ob1.x * i;
    temp.y = ob1.y * i;

    return temp;
}

// Overload * another way.
coord operator*(int i, coord ob2)
{
    coord temp;

    temp.x = ob2.x * i;
    temp.y = ob2.y * i;

    return temp;
}

int main()
{
    coord o1(10, 10), o2;
    int x, y;

    o2 = o1 * 2; // ob * int
    o2.get_xy(x, y);
    cout << "(o1*2) X: " << x << ", Y: " << y << "\n";

    o2 = 3 * o1;
    o2.get_xy(x, y);
    cout << "(3*o1) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

3. By using friend functions, you make it possible to have a built-in type as the left operand. When member functions are used, the left operand must be an object of the class for which the operator is defined.

4.
```
// Overload the -- relative to coord class using a friend.
```

```
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    friend coord operator--(coord &ob); // prefix
    friend coord operator--(coord &ob, int notused); //
       postfix
};

// Overload -- (prefix) for coord class using a friend.
coord operator--(coord &ob)
{
    ob.x--;
    ob.y--;
    return ob;
}

// Overload -- (postfix) for coord class using a friend.
coord operator--(coord &ob, int notused)
{
    ob.x--;
    ob.y--;
    return ob;
}

int main()
{
    coord o1(10, 10);
    int x, y;

    --o1; // decrement an object
    o1.get_xy(x, y);
    cout << "(--o1) X: " << x << ", Y: " << y << "\n";

    o1--;
    o1.get_xy(x, y);
    cout << "(o1--) X: " << x << ", Y: " << y << "\n";

    return 0;
}
```

6.6 EXERCISE

1.
```
#include <iostream>
#include <cstdlib>
```

```
using namespace std;

class dynarray
{
    int *p;
    int size;
public:
    dynarray(int s);
    int &put(int i);
    int get(int i);
    dynarray &operator=(dynarray &ob);
};

// Constructor
dynarray::dynarray(int s)
{
    p = new int [s];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }

    size = s;
}

// Store an element.
int &dynarray::put(int i)
{
    if(i<0 || i>=size)
    {
        cout << "Bounds error!\n";
        exit(1);
    }

    return p[i];
}

// Get an element
int dynarray::get(int i)
{
    if(i<0 || i>=size)
    {
        cout << "Bounds error!\n";
        exit(1);
    }

    return p[i];
}

// Overload = for dynarray
```

```
dynarray &dynarray::operator=(dynarray &ob)
{
    int i;

    if(size!=ob.size)
    {
        cout << "Cannot copy arrays of differing size!\n";
        exit(1);
    }

    for(i = 0; i<size; i++)
        p[i] = ob.p[i];
    return *this;
}

int main()
{
    int i;

    dynarray ob1(10), ob2(10), ob3(100);

    ob1.put(3) = 10;
    i = ob1.get(3);
    cout << i << '\n';

    ob2 = ob1;

    i = ob2.get(3);
    cout << i << '\n';

    // generates an error
    ob1 = ob3; // !!!

    return 0;
}
```

6.7 EXERCISES

1.
```
#include <iostream>
#include <cstring>
#include <cstdlib>
using namespace std;

class strtype
{
    char *p;
    int len;
public:
    strtype(char *s);
    ~strtype()
    {
```

```
        cout << "Freeing " << (unsigned) p << '\n';
        delete [] p;
    }
    char *get() { return p; }
    strtype &operator=(strtype &ob);
    char &operator[](int i);
};

strtype::strtype(char *s)
{
    int l ;

    l = strlen(s)+1;

    p = new char [l];
    if(!p)
    {
        cout << "Allocation error\n";
        exit(1);
    }
    len = l;
    strcpy(p, s);
}

// Assign an object.
strtype &strtype::operator=(strtype &ob)
{
    // see if more memory is needed
    if(len < ob.len) // need to allocate more memory
    {
        delete [] p;
        p = new char (ob.len);
        if(!p)
        {
            cout << "Allocation error\n";
            exit(1);
        }
    }
    len = ob.len;
    strcpy(p, ob.p);
    return *this;
}

// Index characters in string.
char &strtype::operator[](int i)
{
    if(i<0 || i>len-1)
    {
        cout << "\nIndex value of ";
        cout << i << " is out-of-bounds.\n";
        exit(1);
```

```
    }

    return p[i];
}

int main()
{
    strtype a("Hello"), b("There");

    cout << a.get() << '\n';
    cout << b.get() << '\n';

    a = b; // now p is not overwritten

    cout << a.get() << '\n';
    cout << b.get() << '\n';

    // access characters using array indexing
    cout << a[0] << a[1] << a[2] << '\n';

    // assign characters using array indexing
    a[0] = 'X';
    a[1] = 'Y';
    a[2] = 'Z';

    cout << a.get() << '\n';

    return 0;
}
```

2.
```
#include <iostream>
#include <cstdlib>
using namespace std;

class dynarray
{
    int *p;
    int size;
public:
    dynarray(int s);
    dynarray &operator=(dynarray &ob);
    int &operator[](int i);
};

// Constructor
dynarray::dynarray(int s)
{
    p = new int [s];
    if(!p)
    {
        cout << "Allocation error\n";
```

```
        exit(1);
    }

    size = s;
}

// Overload = for dynarray
dynarray &dynarray::operator=(dynarray &ob)
{
    int i;

    if(size!=ob.size)
    {
        cout << "Cannot copy arrays of differing size!\n";
        exit(1);
    }

    for(i = 0; i<size; i++)
        p[i] = ob.p[i];
    return *this;
}

// Overload []
int &dynarray::operator[](int i)
{
    if(i<0 || i>size)
    {
        cout << "\nIndex value of ";
        cout << i << " is out-of-bounds.\n";
        exit(1);
    }
    return p[i];
}

int main()
{
    int i;

    dynarray ob1(10), ob2(10), ob3(100);

    ob1[3] = 10;
    i = ob1[3];
    cout << i << "\n";

    ob2 = ob1;

    i = ob2[3];
    cout << i << "\n";

    // generates an error
    ob1 = ob3; // arrays differ sizes
```

```
    return 0;
}
```

MASTERY SKILLS CHECK: Chapter 6

1.
```
// Overload << and
#include <iostream>
using namespace std;

class coord
{
    int x, y; // coordinate values
public:
    coord() { x=0; y=0; }
    coord(int i, int j) { x=i; y=j; }
    void get_xy(int &i, int &j) { i=x; j=y; }
    coord operator<<(int i);
    coord operator>>(int i);
};

// Overload <<.
coord coord::operator<<(int i)
{
    coord temp;

    temp.x = x << i;
    temp.y = y << i;

    return temp;
}

// Overload >>.
coord coord::operator>>(int i)
{
    coord temp;

    temp.x = x >> i;
    temp.y = y >> i;

    return temp;
}

int main()
{
    coord o1(4, 4), o2;
    int x, y;

    o2 = o1 << 2; // ob << int
    o2.get_xy(x, y);
    cout << "(o1<<2) X: " << x << ", Y: " << y << '\n';
```

```
    o2 = o1 >> 2; // ob >> int
    o2.get_xy(x, y);
    cout << "(o1>>2) X: " << x << ", Y: " << y << '\n';

    return 0;
}
```

2.
```
#include <iostream>
using namespace std;

class three_d
{
    int x, y, z;
public:
    three_d(int i, int j, int k)
    {
        x = i; y = j; z = k;
    }
    three_d() { x=0; y=0; z=0; }
    void get(int &i, int &j, int &k)
    {
        i = x; j = y; k = z;
    }
    three_d operator+(three_d ob2);
    three_d operator-(three_d ob2);
    three_d operator++();
    three_d operator--();
};

three_d three_d::operator+(three_d ob2)
{
    three_d temp;
    temp.x = x + ob2.x;
    temp.y = y + ob2.y;
    temp.z = z + ob2.z;

    return temp;
}

three_d three_d::operator-(three_d ob2)
{
    three_d temp;
    temp.x = x - ob2.x;
    temp.y = y - ob2.y;
    temp.z = z - ob2.z;

    return temp;
}

three_d three_d::operator++()
{
```

```
    x++;
    y++;
    z++;

    return *this;
}

three_d three_d::operator--()
{
    x--;
    y--;
    z--;

    return *this;
}

int main()
{
    three_d o1(10, 10, 10), o2(2, 3, 4), o3;
    int x, y, z;

    o3 = o1 + o2;
    o3.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    o3 = o1 - o2;
    o3.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    ++o1;
    o1.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    --o1;
    o1.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    return 0;
}
```

3.
```
#include <iostream>
using namespace std;

class three_d
{
    int x, y, z;
public:
```

```
    three_d(int i, int j, int k)
    {
        x = i; y = j; z = k;
    }
    three_d() { x=0; y=0; z=0; }
    void get(int &i, int &j, int &k)
    {
        i = x; j = y; k = z;
    }
    three_d operator+(three_d &ob2);
    three_d operator-(three_d &ob2);
    friend three_d operator++(three_d &ob);
    friend three_d operator--(three_d &ob);
};

three_d three_d::operator+(three_d &ob2)
{
    three_d temp;
    temp.x = x + ob2.x;
    temp.y = y + ob2.y;
    temp.z = z + ob2.z;

    return temp;
}

three_d three_d::operator-(three_d &ob2)
{
    three_d temp;
    temp.x = x - ob2.x;
    temp.y = y - ob2.y;
    temp.z = z - ob2.z;

    return temp;
}

three_d operator++(three_d &ob)
{
    ob.x++;
    ob.y++;
    ob.z++;

    return ob;
}

three_d operator--(three_d &ob)
{
    ob.x--;
    ob.y--;
    ob.z--;

    return ob;
```

```
}

int main()
{
    three_d o1(10, 10, 10), o2(2, 3, 4), o3;
    int x, y, z;

    o3 = o1 + o2;
    o3.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    o3 = o1 - o2;
    o3.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    ++o1;
    o1.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    --o1;
    o1.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    return 0;
}
```

4. A binary member operator function is passed the left operand implicitly via **this** pointer. A binary friend operator function is passed both operands explicitly. Unary member operator functions have no explicit parameters. A friend unary operator function has one parameter.

5. You will need to overload the = operator when the default bitwise copy is insufficient. For example, you might have a situation in which you want only parts of the data in one object to be assigned to another object.

6. No.

7.
```
#include <iostream>
using namespace std;

class three_d
{
    int x, y, z;
public:
    three_d(int i, int j, int k)
    {
        x = i; y = j; z = k;
    }
```

```
    three_d() { x=0; y=0; z=0; }
    void get(int &i, int &j, int &k)
    {
        i = x; j = y; k = z;
    }
    friend three_d operator+(three_d ob, int i);
    friend three_d operator+(int i, three_d ob);
};

three_d operator+(three_d ob, int i)
{
    three_d temp;
    temp.x = ob.x + i;
    temp.y = ob.y + i;
    temp.z = ob.z + i;

    return temp;
}

three_d operator+(int i, three_d ob)
{
    three_d temp;
    temp.x = ob.x + i;
    temp.y = ob.y + i;
    temp.z = ob.z + i;

    return temp;
}

int main()
{
    three_d o1(10, 10, 10);
    int x, y, z;

    o1 = o1 + 10;
    o1.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    o1 = -20 + o1;
    o1.get(x, y, z);
    cout << "X: " << x << ", Y: " << y;
    cout << ", Z: " << z << "\n";

    return 0;
}
```

8.
```
#include <iostream>
using namespace std;

class three_d
```

```
{
    int x, y, z;
public:
    three_d(int i, int j, int k)
    {
        x = i; y = j; z = k;
    }
    three_d() { x=0; y=0; z=0; }
    void get(int &i, int &j, int &k)
    {
        i = x; j = y; k = z;
    }
    int operator==(three_d ob2);
    int operator!=(three_d ob2);
    int operator||(three_d ob2);
};

int three_d::operator==(three_d ob2)
{
    return x==ob2.x && y==ob2.y && z==ob2.z;
}

int three_d::operator!=(three_d ob2)
{
    return x!=ob2.x && y!=ob2.y && z!=ob2.z;
}

int three_d::operator||(three_d ob2)
{
    return x||ob2.x && y||ob2.y && z||ob2.z;
}

int main()
{
    three_d o1(10, 10, 10), o2(2, 3, 4), o3(0, 0, 0);

    if(o1==o1)
        cout << "o1==o1\n";

    if(o1!=o2)
        cout << "o1!=o2\n";

    if(o3 || o1)
        cout << "o1 or o3 is true\n";

    return 0;
}
```

9. The [ ] is usually overloaded to allow an array encapsulated within a class to be indexed with the normal array indexing syntax.

CUMULATIVE SKILLS CHECK: Chapter 6

1.
```
/*
   For clarity, no error checking has been used. However
   you should add some if using this code for a real
   application.
*/
#include <iostream>
#include <cstring>
using namespace std;

class strtype
{
    char s[80];
public:
    strtype() { *s = '\0'; }
    strtype(char *p) { strcpy(s, p); }
    char *get() { return s; }
    strtype operator+(strtype s2);
    strtype operator=(strtype s2);
    int operator<(strtype s2);
    int operator>(strtype s2);
    int operator==(strtype s2);
};

strtype strtype::operator+(strtype s2)
{
    strtype temp;

    strcpy(temp.s, s);
    strcat(temp.s, s2.s);

    return temp;
}
strtype strtype::operator=(strtype s2)
{
    strcpy(s, s2.s);

    return *this;
}

int strtype::operator<(strtype s2)
{
    return strcmp(s, s2.s) < 0;
}

int strtype::operator>(strtype s2)
{
    return strcmp(s, s2.s) > 0;
}
```

```
int strtype::operator==(strtype s2)
{
    return strcmp(s, s2.s) == 0;
}

int main()
{
    strtype o1("Hello"), o2("There"), o3;

    o3 = o1 + o2;
    cout << o3.get() << '\n';

    o3 = o1;
    if(o1==o3)
        cout << "o1 equals o3\n";

    if(o1>o2)
        cout << "o1 > o2\n";

    if(o1<o2)
        cout << "o1 < o2\n";\

    return 0;
}
```

REVIEW SKILLS CHECK: Chapter 7

1. No. Overloading an operator simply expands the data types upon which it can operate, but no preexisting operations are affected.

2. Yes. You cannot overload an operator relative to one of C++'s built-in types.

3. No, the precedence cannot be changed. No, the number of operands cannot be altered.

4.
```
#include <iostream>
using namespace std;

class array
{
    int nums[10];
public:
    array();
    void set(int n[10]);
    void show();
    array operator+(array ob2);
    array operator-(array ob2);
    int operator==(array ob2);
};

array::array()
{
    int i;
```